

1.Comparación entre marcos legales: UE vs. EE.UU.

A continuación, una tabla comparativa centrada en **propiedad intelectual del código generado por IA, datos de entrenamiento y responsabilidad**.

1. Propiedad intelectual del contenido generado por IA

Tema	Unión Europea	Estados Unidos
¿La IA puede ser autora?	No. La obra debe ser resultado de una “creación intelectual humana”. Si la IA genera el código casi sola, puede quedar sin protección.	No. La Oficina de Copyright exige autoría humana. Obras generadas íntegramente por IA no pueden registrarse .
Protección del código generado por IA	Solo se protege si hay intervención creativa humana significativa.	Igual: solo se protege la parte creada por humanos.
Riesgo de infracción (p. ej., código copiado por IA)	Si la IA reproduce código protegido sin permiso, el desarrollador que lo usa puede ser responsable.	Igual: reproducir código protegido puede ser infracción, y se discute si el entrenamiento entra en <i>fair use</i> .

2. Uso de datos para entrenar IA (copyright + datos personales)

Tema	Unión Europea	Estados Unidos
Uso de obras protegidas para entrenar IA	Permitido bajo <i>text and data mining</i> (TDM), pero los autores pueden hacer <i>opt-out</i> .	No hay ley federal. Se discute bajo <i>fair use</i> , sin respuesta definitiva.
Transparencia sobre datasets	El AI Act exige documentación, trazabilidad y respeto a derechos de autor.	No hay obligación federal de transparencia.
Datos personales en entrenamiento	GDPR exige base legal, minimización y seguridad estricta.	No existe equivalente federal; depende de leyes estatales como CCPA.

2. Análisis de Herramientas de Auditoría de Código

Conclusión general comparativa

Herramienta	Para qué sirve	Mejor uso	No sirve para
CodeQL	Seguridad y análisis estático	Detectar vulnerabilidades	Licencias, IA
FOSSology	Auditoría legal y licencias	Detectar código copiado o incompatible	Seguridad
Horusec	Seguridad	Vulnerabilidades en código	Licencias, IA
Turnitin	Plagio y detección de IA	Documentación	Código

1. CodeQL (GitHub)

Conclusión: Excelente para seguridad y calidad del código, pero no para auditoría legal o detección de contenido generado por IA.

2. FOSSology (Linux Foundation)

Conclusión: Ideal para auditoría legal, especialmente cuando se usa IA generativa o se sospecha de código copiado.

3. Horusec (Seguridad)

Conclusión: Excelente para seguridad, especialmente para proyectos con riesgos como SQL injection o contraseñas inseguras (como tu ejemplo).

4. Turnitin (Documentación y texto)Conclusión

Conclusión: Perfecto para documentación, no para auditoría de código.

Recomendación final

Para un proyecto donde se usa IA generativa, la combinación más útil es:

❑ **FOSSology + CodeQL/Horusec** (legal + seguridad)

3.Auditoría del proyecto “CodeHelper AI” – Lista de Hallazgos

A continuación se presentan los hallazgos detectados en el código proporcionado, clasificados por nivel de riesgo.

1. Riesgos de Licencias (Copyright)

Hallazgo	Descripción	Riesgo
Código copiado de StackOverflow sin atribución	La función quicksort() está copiada directamente de StackOverflow sin respetar la licencia CC BY-SA 4.0 , que exige atribución obligatoria y compartir bajo la misma licencia.	ALTO
Código generado por IA sin revisión ni atribución	Varias funciones están marcadas como “Generado por IA sin validaciones”. Esto puede incluir fragmentos aprendidos de código protegido.	MEDIO
Ausencia de licencia del proyecto	No se declara ninguna licencia en el repositorio, lo que genera ambigüedad legal y dificulta el cumplimiento.	MEDIO

2. Vulnerabilidades de Seguridad (introducidas por IA o malas prácticas)

Hallazgo	Descripción	Riesgo
SQL Injection en get_user_tasks()	Uso de interpolación directa: f"SELECT * FROM tasks WHERE user = '{username}'". Permite inyección SQL.	ALTO
Uso de MD5 para contraseñas	hashlib.md5() es criptográficamente inseguro. Facilita ataques de fuerza bruta y rainbow tables.	ALTO
Debug activado en producción	app.run(debug=True) expone información sensible y permite ejecución remota de código en ciertos casos.	ALTO
Llamada a API externa sin validación ni manejo de errores	requests.get() sin clave de API, sin control de excepciones, sin verificar origen ni legalidad del contenido.	MEDIO
Secret key generada dinámicamente en cada ejecución	app.secret_key = os.urandom(16) invalida sesiones en cada reinicio y no es adecuada para producción.	MEDIO
Falta de sanitización de inputs	Los valores de formularios se usan directamente sin validación.	MEDIO
Estructura del código	Código desordenado, imports dentro de	BAJO

Hallazgo	Descripción	Riesgo
inconsistente y mezclada	funciones, duplicación de funciones. Puede generar fallos inesperados.	

3. Falta de Atribución, Documentación y Buenas Prácticas

Hallazgo	Descripción	Riesgo
Código copiado sin atribución (StackOverflow)	La función quicksort viola la licencia CC BY-SA.	ALTO
Comentarios que indican generación por IA sin revisión	“Generado por IA sin validaciones” implica riesgo de errores o contenido no verificado.	MEDIO
Falta de documentación de API externa (Unsplash)	No se explica por qué se usa, ni se incluye clave de API, ni se menciona su licencia.	MEDIO
Funciones incompletas o mal definidas	Ejemplo: def add_user(username, password, email): está incompleta.	BAJO
Imports mezclados y desordenados	Dificulta mantenimiento y auditoría.	BAJO

4. Política Interna para el Uso de IA Generativa en el Desarrollo de Software

Versión 1.0 – Departamento de Ingeniería Fecha de emisión: 18/02/2026

1. Propósito

Esta política establece las normas y procedimientos para el uso responsable de herramientas de **IA generativa** en el desarrollo de software dentro de la organización. Su objetivo es garantizar:

- La calidad, seguridad y trazabilidad del código.
- El cumplimiento de licencias y derechos de autor.
- La protección de datos confidenciales y propiedad intelectual.
- La prevención de riesgos legales, técnicos y operativos.

2. Alcance

Esta política aplica a:

- Todo el personal técnico: desarrolladores, QA, DevOps, arquitectos y analistas.
- Consultores externos que participen en proyectos de software.

- Herramientas de IA generativa como ChatGPT, GitHub Copilot, CodeWhisperer, Claude, etc.
- Todo el ciclo de vida del software: diseño, codificación, documentación, pruebas y mantenimiento.

3. Directrices de Documentación

3.1. Registro del uso de IA

Cada vez que se utilice IA generativa para producir código, documentación o ideas técnicas, se debe registrar:

- Herramienta utilizada.
- Fecha de generación.
- Fragmento generado.
- Revisión humana realizada.

Ejemplo de comentario obligatorio en código:

```
python
# Fragmento generado con IA (ChatGPT) el 18/02/2026. Revisado y validado
manualmente.
```

3.2. Documentación en repositorios

En cada commit que incluya contenido generado por IA se debe indicar:

- “AI-assisted commit” o equivalente.
- Descripción clara de qué parte fue generada por IA.

3.3. Prohibiciones

- No se permite integrar código generado por IA sin revisión humana.
- No se permite introducir código externo sin atribución.
- No se permite usar IA para generar documentación legal o contractual sin revisión del área correspondiente.

4. Proceso de Revisión y Validación

4.1. Revisión técnica obligatoria

Todo código generado por IA debe pasar por:

- **Revisión manual del desarrollador responsable.**
- **Revisión por pares (peer review).**

- **Análisis automatizado** con herramientas como:
- CodeQL
- Horusec
- SonarQube

4.2. Validación de seguridad

El revisor debe verificar:

- Ausencia de vulnerabilidades (SQL injection, XSS, CSRF, contraseñas inseguras, etc.).
- Cumplimiento de estándares internos de seguridad.
- No uso de funciones criptográficas obsoletas (ej. MD5).

4.3. Validación funcional

- El código debe ser probado manualmente y mediante pruebas automatizadas.
- Se debe confirmar que no existen “alucinaciones” o comportamientos no documentados.

5. Gestión de Licencias y Derechos de Autor

5.1. Reglas generales

- Todo código externo debe incluir su licencia correspondiente.
- Está prohibido integrar código con licencias incompatibles con el proyecto.
- Se debe verificar la compatibilidad de licencias antes de integrar dependencias.

5.2. Código generado por IA

- Se debe asumir que el código generado puede contener fragmentos aprendidos de obras protegidas.
- Por ello, **es obligatorio analizarlo con herramientas como FOSSology**.
- Si se detecta coincidencia con código protegido, debe reescribirse o eliminarse

5.3. Atribución

Cuando se utilice contenido de terceros (StackOverflow, GitHub, documentación pública):

- Se debe incluir la atribución requerida.
- Se deben respetar las condiciones de la licencia (ej. CC BY-SA exige compartir igual).

6. Protocolo ante Incidentes

6.1. Tipos de incidentes cubiertos

- Filtración de datos sensibles a herramientas de IA.
- Inclusión de código plagiado o con licencia incompatible.
- Vulnerabilidades críticas introducidas por IA generativa.
- Uso indebido de herramientas de IA por parte del personal.

6.2. Procedimiento de respuesta

- **Notificación inmediata** al responsable de seguridad o líder técnico.
- **Aislamiento del código afectado** (rollback, bloqueo del commit o rama).

Análisis forense del incidente:

1. Herramienta utilizada.
2. Datos enviados.
3. Fragmentos comprometidos.

Corrección del problema:

4. Reescritura del código
5. Eliminación de contenido protegido.
6. Aplicación de parches de seguridad.

Informe final con:

7. Causa raíz.
8. Impacto.
9. Medidas preventivas.

6.3. Medidas disciplinarias

El incumplimiento reiterado de esta política puede resultar en:

10. Restricción del acceso a herramientas de IA.
11. Sanciones internas según el reglamento de la empresa.
12. Notificación al área legal en casos graves.

7. Buenas Prácticas Recomendadas

13. Usar IA como apoyo, no como sustituto del criterio técnico
14. Evitar introducir datos sensibles en prompts.

15. Mantener un registro claro del origen del código.
16. Revisar periódicamente las políticas de uso de cada herramienta de IA.
17. Priorizar siempre la seguridad y la trazabilidad.

8. Revisión y Actualización

18. Esta política será revisada cada **12 meses** o cuando:
19. Se adopten nuevas herramientas de IA.
20. Cambien las regulaciones legales aplicables.
21. Se detecten incidentes relevantes.

5. Debate y Reflexión Ética

"¿Debe permitirse la IA generativa en software crítico como salud, finanzas o defensa?"

La IA generativa ofrece enormes ventajas: acelera el desarrollo, detecta errores, analiza grandes volúmenes de código y permite simular escenarios complejos. En sectores como la salud o la banca, esto puede traducirse en diagnósticos más rápidos o sistemas financieros más seguros.

Pero también existen riesgos importantes. La IA generativa puede inventar información con total seguridad. Ya hemos visto casos reales: chatbots médicos que dieron recomendaciones peligrosas, o sistemas de contratación como el de Amazon que discriminaban a mujeres por sesgos en los datos.

Además, la IA generativa es difícil de auditar. En software crítico, cada decisión debe ser explicable, y los modelos actuales funcionan como cajas negras. También pueden generar código inseguro, lo que en defensa o finanzas puede tener consecuencias graves.

Entonces, ¿debería permitirse? La respuesta más razonable es: **sí, pero con condiciones estrictas**. Supervisión humana obligatoria, auditorías continuas, validación independiente y prohibición de decisiones autónomas sin revisión.

La clave no es prohibir la tecnología, sino **regularla, controlarla y usarla con responsabilidad**. En sectores donde está en juego la vida humana, la estabilidad financiera o la seguridad nacional, la IA debe ser una herramienta poderosa, pero nunca un sustituto del juicio humano.

ALUMNO.

ANGEL ECHENIQUE

Z0271067V

DICAMPUS / EDU-RUS (RUSGAR)

18/02/26